

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA ĐÀO TẠO SAU ĐẠI HỌC**



**BÁO CÁO TIỂU LUẬN
MÔN HỌC CÁC HỆ THỐNG PHÂN TÁN**

ĐỀ TÀI:

**XÂY DỰNG HỆ THỐNG CHAT NGANG HÀNG
PEER-TO-PEER**

Giảng viên: TS. Kim Ngọc Bách

Học viên: B25CHHT112 - Đào Văn Tâm

B25CHHT117 - Lâm Thành Trung

B25CHHT119 - Nguyễn Xuân Tùng

Lớp: M25CQHT02-B

Nhóm: 08

Hà Nội, ngày 20 tháng 05 năm 2026

TÓM TẮT NỘI DUNG

Trong bối cảnh công nghệ mạng phát triển mạnh mẽ, các hệ thống phân tán đóng vai trò then chốt trong việc tạo ra một tập hợp các máy tính độc lập nhưng xuất hiện trước người dùng như một thể thống nhất và trong suốt. Báo cáo này trình bày quá trình nghiên cứu, thiết kế và cài đặt hệ thống chat ngang hàng mang tên **PeerBuzz** dựa trên Framework Flutter nhằm giải quyết bài toán giao tiếp phi tập trung. Hệ thống áp dụng mô hình kiến trúc Hybrid P2P, kết hợp giữa tính tự trị của các nút mạng và sự hỗ trợ điều phối danh sách thành viên từ một Bootstrap Server đơn giản.

Về mặt kỹ thuật, hệ thống PeerBuzz thực hiện truyền thông tin cậy thông qua giao thức TCP Socket, đảm bảo các thông điệp được chuyển giao đến đúng đích và nguyên vẹn. Quá trình đóng gói dữ liệu được chuẩn hóa dưới định dạng JSON để đảm bảo tính tương thích và mở rộng của hệ thống. Một trong những đóng góp học thuật trọng tâm của tiểu luận là việc triển khai thuật toán nhãn thời gian logic Lamport nhằm thiết lập quan hệ “xảy ra trước” giữa các sự kiện chat, giúp duy trì tính nhất quán về thứ tự tin nhắn trên toàn mạng phủ mà không cần đồng bộ đồng hồ vật lý.

Nhằm đối phó với hiện tượng biến động nút mạng và các lỗi sập đồ thường gặp trong môi trường phân tán, hệ thống được trang bị cơ chế phát hiện lỗi thông qua Heartbeat và thời gian quá hạn. Kết quả thực nghiệm cho thấy hệ thống PeerBuzz đạt được hiệu năng ổn định trong việc xử lý đồng thời nhiều kết nối và đảm bảo tính sẵn sàng cao của dịch vụ ngay cả khi có một số nút rời mạng đột ngột. Tiểu luận không chỉ minh chứng cho khả năng vận dụng các nguyên lý cốt lõi của hệ phân tán vào thực tiễn mà còn mở ra hướng phát triển cho các ứng dụng thời gian thực quy mô lớn.

LỜI CẢM ƠN

Lời đầu tiên, với tất cả sự trân trọng và lòng biết ơn, nhóm thực hiện tiểu luận xin được gửi lời cảm ơn chân thành và sâu sắc nhất tới *TS. Kim Ngọc Bách*, giảng viên bộ môn Các hệ thống phân tán tại Khoa Đào tạo Sau đại học, Học viện Công nghệ Bưu chính Viễn thông. Trong suốt quá trình học tập và nghiên cứu, chúng em đã may mắn được tiếp nhận những bài giảng tâm huyết và đầy trí tuệ của Thầy. Không chỉ dừng lại ở những lý thuyết kinh điển về hệ thống phân tán, Thầy đã truyền cảm hứng và cung cấp cho chúng em một cái nhìn toàn diện, hiện đại về các đặc tính cốt lõi như tính trong suốt, tính mở, khả năng mở rộng và đặc biệt là tính chịu lỗi trong môi trường mạng không tin cậy. Những kiến thức nền tảng mà Thầy truyền đạt về mô hình Servant, cơ chế Peer Discovery và tư duy giải quyết bài toán nhất quán dữ liệu bằng thời gian logic đã trở thành kim chỉ nam giúp chúng em định hình và triển khai thành công hệ thống PeerBuzz. Nếu không có sự hướng dẫn tận tâm và những gợi mở mang tính định hướng của Thầy, tiểu luận này chắc chắn sẽ không thể hoàn thiện với chiều sâu học thuật như hiện tại.

Nhóm cũng xin gửi lời cảm ơn chân thành tới Ban lãnh đạo Khoa Đào tạo Sau đại học cùng các phòng ban chức năng của Học viện Công nghệ Bưu chính Viễn thông. Nhà trường đã tạo mọi điều kiện thuận lợi về hạ tầng, môi trường học tập và cung cấp nguồn học liệu quý báu để chúng em có thể nghiên cứu và thực hành một cách bài bản nhất.

Cuối cùng, mặc dù đã dành rất nhiều tâm huyết và nỗ lực để áp dụng các nguyên lý khoa học vào thực tiễn, nhưng do giới hạn về mặt kinh nghiệm, tiểu luận chắc chắn vẫn còn những điểm cần cải thiện. Chúng em kính mong nhận được những ý kiến phê bình, chỉ bảo quý báu từ Thầy để chúng em có thêm cơ hội hoàn thiện tri thức, tiếp tục vững bước trên con đường nghiên cứu và ứng dụng công nghệ phân tán trong tương lai.

Một lần nữa, *chúng em xin chân thành cảm ơn!*

MỤC LỤC

TÓM TẮT NỘI DUNG	1
LỜI CẢM ƠN	2
MỤC LỤC.....	3
ĐỊNH NGHĨA VIẾT TẮT	5
DANH MỤC HÌNH VẼ	7
DANH MỤC BẢNG BIỂU	8
Chương 1: Giới thiệu.....	9
1.1. Tổng quan về đề tài	9
1.2. Đặt vấn đề.....	9
Chương 2: Một số khái niệm nền tảng	11
2.1. Mô hình Hybrid P2P	11
2.2. Truyền thông giữa các tiến trình qua TCP Socket.....	12
2.2.1. Khái niệm về Socket và giao thức TCP	12
2.2.2. Các hàm nguyên thủy của Socket.....	12
2.2.3. Vận dụng TCP Socket trong hệ thống PeerBuzz.....	13
2.3. Đóng gói dữ liệu	13
2.3.1. Khái niệm Marshalling và Unmarshalling.....	14
2.3.2. Thách thức về tính không đồng nhất	14
2.3.3. Giải pháp đóng gói trong dự án PeerBuzz	14
2.4. Đồng hồ Logic Lamport.....	15
2.5. Tính chịu lỗi và bài toán Churn	17
Chương 3: Xây dựng hệ thống.....	19
3.1. Kiến trúc hệ thống.....	19
3.2. Giao thức trao đổi thông điệp	20
3.2.1. Quy trình trao đổi tin nhắn trực tiếp 1 - 1	22
3.2.2. Quy trình trao đổi tin nhắn nhóm.....	23

3.2.3. Thông điệp điều khiển và xử lý rời mạng.....	23
3.3. Cơ chế Peer Discovery	24
3.4. Xử lý lỗi và thử nghiệm hệ thống	26
3.4.1. Các mô hình lỗi và giải pháp xử lý.....	26
3.4.2. Thử nghiệm hệ thống.....	27
<i>Chương 4: Kết quả và Đánh giá.....</i>	<i>29</i>
4.1. Giao diện ứng dụng	29
4.2. Các kịch bản thử nghiệm	33
4.3. Phân tích kết quả và hạn chế.....	34
<i>TÀI LIỆU THAM KHẢO</i>	<i>37</i>

ĐỊNH NGHĨA VIẾT TẮT

Từ viết tắt	Từ đầy đủ	Mô tả
API	Application Programming Interface	Giao diện lập trình ứng dụng cung cấp các công cụ hỗ trợ
P2P	Peer-to-Peer	Mô hình mạng ngang hàng trong đó các nút mạng vừa đóng vai trò máy khách vừa đóng vai trò máy chủ
TCP	Transmission Control Protocol	Giao thức hướng kết nối ở tầng vận tải, đảm bảo truyền dữ liệu tin cậy, đúng thứ tự và toàn vẹn
IP	Internet Protocol	Giao thức định tuyến ở tầng mạng, cung cấp địa chỉ định danh cho mỗi thiết bị trên Internet
JSON	JavaScript Object Notation	Định dạng văn bản nhẹ dùng để trao đổi dữ liệu giữa các hệ thống không đồng nhất
WAN	Wide Area Network	Mạng diện rộng kết nối các thiết bị qua khoảng cách địa lý lớn, điển hình là Internet
LAN	Local Area Network	Mạng nội bộ kết nối các thiết bị trong phạm vi hẹp như văn phòng hoặc gia đình
NAT	Network Address Translation	Kỹ thuật chuyển đổi địa chỉ mạng cho phép nhiều thiết bị dùng chung một địa chỉ IP công cộng
RFC	Request for Comments	Tài liệu đặc tả tiêu chuẩn kỹ thuật do IETF phát hành về các giao thức Internet

IETF	Internet Engineering Task Force	Tổ chức quốc tế chịu trách nhiệm phát triển và chuẩn hóa các giao thức Internet
UTF-8	Unicode Transformation Format - 8 bit	Chuẩn mã hóa ký tự đa ngôn ngữ phổ biến, sử dụng độ dài byte thay đổi từ 1 đến 4 byte
ACK	Acknowledgement	Gói tin xác nhận trong giao thức TCP, dùng để báo nhận thành công và đảm bảo truyền lại khi mất gói
ID	Identifier	Định danh duy nhất dùng để phân biệt các thực thể như Peer, tin nhắn trong hệ thống

DANH MỤC HÌNH VẼ

Hình 1: Sơ đồ kiến trúc hệ thống	19
Hình 2: Luồng giao tiếp giữa hai Peer chat 1 - 1	22
Hình 3: Luồng giao tiếp giữa Peer và Bootstrap Server trong pha Discovery	24
Hình 4: Màn hình đăng nhập Peer trên thiết bị di động	29
Hình 5: Danh sách Peer trực tuyến hiển thị trên thiết bị di động	30
Hình 6: Giao diện thông tin Peer trên thiết bị di động	30
Hình 7: Giao diện chat 1 - 1 trên thiết bị di động	31
Hình 8: Màn hình đăng nhập Peer trên nền tảng Desktop	31
Hình 9: Màn hình thông tin Peer trên nền tảng Desktop	32
Hình 10: Màn hình chat 1 - 1 trên Desktop cùng tin nhắn gửi File	32
Hình 11: Màn hình chat 1 - 1 trên nền tảng Desktop	33

DANH MỤC BẢNG BIỂU

Bảng 1: Đoạn mã thiết lập kết nối TCP từ Peer	20
Bảng 2: Định nghĩa các loại thông điệp trong giao thức.....	21
Bảng 3: Cấu trúc thông điệp chat có gắn nhãn Lamport.....	22
Bảng 4: Xử lý các loại thông điệp tại Bootstrap Server.....	26
Bảng 5: Môi trường thử nghiệm hệ thống.....	27
Bảng 6: Các kịch bản thử nghiệm và kết quả mong đợi	27
Bảng 7: Kết quả thực nghiệm các kịch bản.....	33

Chương 1: Giới thiệu

1.1. Tổng quan về đề tài

Trong bối cảnh bùng nổ của công nghệ thông tin, hệ thống phân tán đã trở thành nền tảng cốt lõi giúp kết nối hàng triệu thiết bị độc lập thành một thể thống nhất. Theo định nghĩa kinh điển của Tanenbaum [1], hệ thống phân tán là một tập hợp các thành phần tính toán tự trị, xuất hiện trước người dùng như một hệ thống gắn kết và đồng nhất. Đặc tính quan trọng nhất của mô hình này là khả năng cộng tác giữa các nút mạng độc lập để đạt được mục tiêu chung thông qua việc trao đổi thông điệp. Trong lĩnh vực truyền thông thời gian thực, sự chuyển dịch kiến trúc từ tập trung sang phi tập trung đang diễn ra mạnh mẽ nhằm tối ưu hóa tài nguyên mạng. Đề tài “*Xây dựng hệ thống chat ngang hàng P2P*” của nhóm tập trung vào việc xây dựng ứng dụng PeerBuzz dựa trên mô hình mạng ngang hàng. Điểm khác biệt cốt lõi của hệ thống này nằm ở kiến trúc nút mạng: Mỗi thành phần tham gia không chỉ đóng vai trò là thực thể yêu cầu dịch vụ - Client mà còn đồng thời hoạt động như một máy chủ cung cấp dịch vụ - Server, hay còn gọi là mô hình Servant. Các Peer sẽ thiết lập một mạng phủ logic bên trên hạ tầng Internet vật lý để giao tiếp trực tiếp với nhau. Về mặt kỹ thuật, PeerBuzz được phát triển trên nền tảng Flutter [7] đa nền tảng, cho phép triển khai đồng bộ trên Mac, Windows, Android và iOS. Hệ thống vận dụng các nguyên lý kỹ thuật phân tán chuyên sâu như:

- **Truyền thông Socket TCP:** Sử dụng giao thức hướng kết nối để đảm bảo tính tin cậy, tính toàn vẹn và thứ tự của tin nhắn trong quá trình truyền tải
- **Marshalling:** Chuẩn hóa thông điệp qua định dạng JSON để chuyển đổi cấu trúc dữ liệu thành chuỗi byte, đảm bảo tính mở và khả năng tương tác giữa các thiết bị không đồng nhất
- **Kiến trúc Hybrid P2P:** Kết hợp giữa tính phi tập trung của luồng dữ liệu và một máy chủ Bootstrap đơn giản để hỗ trợ quá trình khám phá nút mạng ban đầu
- **Xử lý đồng thời:** Vận dụng cơ chế đa luồng và lập trình bất đồng bộ để mỗi nút mạng có thể xử lý nhiều kết nối TCP cùng lúc mà không gây phong tỏa tiến trình chính

1.2. Đặt vấn đề

Mặc dù các hệ thống chat dựa trên kiến trúc Client - Server truyền thống rất phổ biến, chúng vẫn đối mặt với những hạn chế cố hữu về mặt cấu trúc. Khi số lượng người dùng tăng lên quy mô hàng triệu, máy chủ trung tâm dễ dàng trở thành nút cổ chai hiệu năng.

Bên cạnh đó, mô hình tập trung tiềm ẩn rủi ro về điểm lỗi duy nhất, chỉ cần máy chủ gặp sự cố, toàn bộ mạng lưới giao tiếp sẽ bị tê liệt.

Từ thực tế đó, dự án PeerBuzz đặt ra mục tiêu giải quyết các thách thức kỹ thuật cụ thể sau:

- **Bài toán Peer Discovery:** Trong môi trường Internet với địa chỉ IP động và các lớp tường lửa, làm thế nào để một Peer mới gia nhập có thể tìm thấy địa chỉ các láng giềng mà không cần một cơ sở dữ liệu tập trung duy nhất duy trì mọi kết nối?
- **Quản lý sự thay đổi nút:** Hệ thống phân tán thường có tính động cao, các Peer có thể gia nhập hoặc rời mạng bất kỳ lúc nào. Việc duy trì danh sách các nút đang hoạt động và đảm bảo tin nhắn không bị thất lạc khi một nút đột ngột ngắt kết nối là một thách thức lớn về tính chịu lỗi
- **Tính trong suốt về phân phối:** Làm thế nào để che giấu sự phức tạp của hạ tầng mạng bên dưới như: địa chỉ IP, cổng Socket, mạng phủ để người dùng có cảm giác đang sử dụng một dịch vụ gắn kết duy nhất?
- **Đồng bộ hóa không có đồng hồ chung:** Trong hệ thống phân tán, mỗi máy tính có một bộ đếm thời gian riêng dẫn đến sai lệch thời gian vật lý. Vấn đề đặt ra là thiết lập một cơ chế thời gian logic để đảm bảo các tin nhắn trong hội thoại nhóm được hiển thị theo đúng trình tự trước - sau trên tất cả các thiết bị

Thông qua việc phát triển hệ thống PeerBuzz, nhóm hướng tới việc thực nghiệm hóa các lý thuyết về loại trừ tương hỗ, đồng bộ trạng thái và chịu lỗi để xây dựng một giải pháp giao tiếp an toàn, bảo mật và không phụ thuộc vào máy chủ trung gian, đáp ứng các tiêu chuẩn khắt khe về kỹ thuật trong môn học Các hệ thống phân tán.

Chương 2: Một số khái niệm nền tảng

2.1. Mô hình Hybrid P2P

Trong kiến trúc của các hệ thống phân tán, mô hình ngang hàng lai - Hybrid P2P được định nghĩa là sự kết hợp linh hoạt giữa các thành phần tập trung và các thành phần phi tập trung. Mô hình này ra đời nhằm khắc phục những hạn chế về khả năng tìm kiếm của mạng P2P thuần túy, đồng thời giảm thiểu rủi ro về điểm lỗi duy nhất và nút cổ chai của mô hình Client - Server truyền thống.

Trong hệ thống PeerBuzz, mô hình Hybrid P2P được triển khai cụ thể thông qua hai thành phần cốt lõi:

Bootstrap Server: Server đóng vai trò là thực thể trung tâm đơn giản, hỗ trợ các tiến trình khởi tạo và quản lý mạng phủ

- Chức năng quản lý danh sách: Nhiệm vụ chính của Bootstrap Server là lưu trữ và duy trì danh sách địa chỉ IP và cổng TCP của tất cả các Peer đang hoạt động trong hệ thống.
- Hỗ trợ Peer Discovery: Khi một Peer mới gia nhập mạng, nó liên lạc với Bootstrap Server để đăng ký thông tin cá nhân và nhận về danh sách các Peer khác hiện có. Cơ chế này giúp giải quyết bài toán tìm kiếm thực thể trong môi trường mạng WAN - nơi mà các kỹ thuật như Broadcasting/Multicasting thường không hiệu quả.
- Tính trong suốt về vị trí: Bằng cách cung cấp một điểm truy cập cố định để lấy thông tin láng giềng, Bootstrap Server giúp người dùng cuối cảm thấy như đang sử dụng một dịch vụ duy nhất mà không cần quan tâm đến địa chỉ IP vật lý phức tạp của các nút khác.

Peer Node: Đây là thành phần chính thực thi các nghiệp vụ chat, hoạt động theo mô hình Servant

- Vai trò máy chủ: Mỗi Peer phải mở một cổng TCP Socket để lắng nghe và chấp nhận các kết nối đến từ các láng giềng khác trong mạng phủ.
- Vai trò máy khách: Peer chủ động thiết lập kết nối TCP tới các địa chỉ IP nhận được từ Bootstrap Server để gửi yêu cầu trao đổi thông tin.
- Truyền tin trực tiếp: Sau giai đoạn khám phá ban đầu, việc trao đổi tin nhắn diễn ra trực tiếp giữa các Peer mà không thông qua Bootstrap Server. Điều này giúp tối ưu hóa băng thông tại biên và đảm bảo tính riêng tư của nội dung hội thoại.

Ưu điểm của mô hình Hybrid P2P trong hệ thống PeerBuzz: Việc vận dụng kiến trúc Hybrid P2P mang lại các lợi ích quan trọng trong bám sát lý thuyết hệ phân tán:

1. **Khả năng mở rộng:** Do máy chủ trung tâm Bootstrap chỉ giữ vai trò hỗ trợ kết nối ban đầu, không tham gia vào luồng dữ liệu chat, nên hệ thống không gặp phải vấn đề nút cổ chai hiệu năng khi quy mô người dùng tăng lên.
2. **Xử lý bài toán Churn:** Trong hệ thống phân tán, các nút thường xuyên gia nhập và rời mạng đột ngột. Mô hình lai cho phép Bootstrap Server cập nhật trạng thái Online/Offline theo thời gian thực một cách tập trung, giúp danh sách bạn bè của người dùng luôn chính xác.
3. **Tính bền bỉ:** Nếu Bootstrap Server tạm thời gặp sự cố, các phiên chat đang diễn ra giữa các Peer vẫn có thể tiếp tục hoạt động bình thường nhờ tính chất phi tập trung của luồng dữ liệu.

2.2. Truyền thông giữa các tiến trình qua TCP Socket

Trong các hệ thống phân tán, các tiến trình thường chạy trên các máy tính khác nhau và không dùng chung bộ nhớ, do đó việc trao đổi thông tin phải dựa hoàn toàn vào phương pháp chuyển thông điệp. TCP Socket đóng vai trò là API quan trọng nhất để thiết lập kênh truyền thông này.

2.2.1. Khái niệm về Socket và giao thức TCP

Socket: Được định nghĩa là một điểm kết cuối truyền thông mà một ứng dụng có thể ghi dữ liệu vào để gửi đi hoặc đọc dữ liệu đến từ mạng. Socket trừu tượng hóa các cổng và địa chỉ IP, tạo thành một định danh duy nhất cho tiến trình trên mạng.

Giao thức TCP [3]: Là một giao thức hướng kết nối, đảm bảo tính tin cậy cao thông qua cơ chế bắt tay ba bước để thiết lập liên kết. TCP tự động xử lý việc bảo toàn thứ tự dữ liệu, phát hiện và sửa lỗi nếu có gói tin bị mất trong quá trình truyền tải.

2.2.2. Các hàm nguyên thủy của Socket

Để thực hiện truyền thông, hệ điều hành cung cấp các hàm cơ bản mà cả máy khách và máy chủ phải tuân thủ [4]:

- *socket()*: Khởi tạo một điểm kết cuối truyền thông mới
- *bind()*: Gắn một địa chỉ IP cục bộ và số hiệu cổng vào socket (thường dùng cho phía Server)
- *listen()*: Thông báo trạng thái sẵn sàng tiếp nhận các yêu cầu kết nối và thiết lập hàng đợi cho các yêu cầu đang chờ

- *accept()*: Một lời gọi phong tỏa trên Server để đợi một yêu cầu kết nối đến. Khi thành công, nó trả về một socket mới dành riêng cho phiên liên lạc đó
- *connect()*: Phía Client sử dụng hàm này để chủ động thiết lập kết nối tới địa chỉ và cổng của Server
- *send()*, *recv()*: Các hàm dùng để gửi và nhận chuỗi byte dữ liệu qua kênh truyền đã thiết lập
- *close()*: Giải phóng liên kết và tài nguyên mạng sau khi hoàn thành trao đổi.

2.2.3. Vận dụng TCP Socket trong hệ thống PeerBuzz

Trong dự án PeerBuzz, việc sử dụng TCP Socket là lựa chọn cốt lõi để đảm bảo tính ổn định của hệ thống chat ngang hàng:

- Mô hình Servant: Do tính chất P2P, mỗi Peer không chỉ là Client mà còn là một Server. Vì vậy, mỗi Peer phải thực hiện đồng thời cả hai vai trò: mở một cổng TCP để *listen* và *accept* các kết nối từ bạn bè, đồng thời sử dụng *connect* để chủ động bắt đầu phiên chat với các Peer khác.
- Truyền tin đáng tin cậy: Hệ thống yêu cầu tin nhắn phải được truyền thành công và đúng thứ tự tới đích. TCP Socket tự động giải quyết các lỗi mất gói tin thông qua cơ chế xác nhận - ACKs và truyền lại, giúp tăng ứng dụng giảm bớt sự phức tạp khi xử lý lỗi mạng.
- Marshalling dữ liệu: Vì Socket chỉ truyền các chuỗi byte, hệ thống sử dụng định dạng JSON để đóng gói cấu trúc tin nhắn gồm: người gửi, nội dung, nhãn thời gian trước khi gửi qua Socket, đảm bảo các nút mạng chạy trên các nền tảng khác nhau như Mac, Windows, Android, iOS đều có thể hiểu được dữ liệu của nhau. Việc sử dụng TCP Socket trực tiếp thông qua thư viện *dart:io* của *Flutter* giúp hệ thống PeerBuzz đạt được hiệu năng cao và khả năng kiểm soát luồng dữ liệu chặt chẽ, đáp ứng yêu cầu của một hệ thống phân tán chuyên nghiệp.

2.3. Đóng gói dữ liệu

Trong hệ thống phân tán, do các tiến trình không dùng chung bộ nhớ, mọi việc trao đổi thông tin đều phải thông qua cơ chế chuyển thông điệp. Tuy nhiên, một thách thức lớn nảy sinh là làm thế nào để bên nhận có thể hiểu chính xác ý nghĩa của chuỗi byte mà bên gửi truyền đi. Quá trình giải quyết vấn đề này được gọi là đóng gói và biểu diễn dữ liệu.

2.3.1. Khái niệm Marshalling và Unmarshalling

Marshalling - Đóng gói: Là quá trình chuyển đổi các cấu trúc dữ liệu hoặc đối tượng trong bộ nhớ của tiến trình gửi thành một định dạng phù hợp để truyền tải qua mạng, thường là một chuỗi byte liên tục. Quá trình này bao gồm việc thu thập các tham số và mã hóa chúng theo một quy tắc thống nhất.

Unmarshalling - Giải gói: Là quá trình ngược lại diễn ra tại phía nhận. Hệ thống sẽ bóc tách chuỗi byte nhận được từ kênh truyền và tái tạo lại thành cấu trúc dữ liệu ban đầu trong không gian địa chỉ của tiến trình nhận để ứng dụng có thể xử lý.

2.3.2. Thách thức về tính không đồng nhất

Việc đóng gói dữ liệu không đơn thuần là sao chép bộ nhớ vì các hệ thống phân tán thường bao gồm các máy tính không đồng nhất:

- Thứ tự byte: Các máy tính có thể sử dụng định dạng Little-endian hoặc Big-endian dẫn đến việc sắp xếp các byte của cùng một số nguyên bị ngược nhau trong bộ nhớ.
- Biểu diễn kiểu dữ liệu: Các bộ vi xử lý và ngôn ngữ lập trình khác nhau có thể có cách biểu diễn số thực, ký tự Unicode và ASCII hoặc cấu trúc dữ liệu khác nhau.
- Cấu trúc dữ liệu phức tạp: Các đối tượng có con trỏ hoặc các cấu trúc lồng nhau đòi hỏi cơ chế đóng gói thông minh hơn để đảm bảo tính toàn vẹn thông tin khi di chuyển qua mạng.

2.3.3. Giải pháp đóng gói trong dự án PeerBuzz

Để đảm bảo tính mở và khả năng tương tác giữa các nền tảng khác nhau, dự án PeerBuzz áp dụng các kỹ thuật đóng gói hiện đại sau:

- Sử dụng định dạng JSON: Thay vì truyền dữ liệu nhị phân thô, hệ thống sử dụng JSON [5] làm định dạng trung gian để đóng gói dữ liệu. JSON là một định dạng văn bản độc lập với ngôn ngữ và nền tảng, giúp giải quyết triệt để vấn đề khác biệt về kiến trúc phần cứng.
- Quy trình xử lý: Dữ liệu tin nhắn bao gồm: ID người gửi, nội dung, nhãn thời gian Lamport được chuyển thành một đối tượng trong ngôn ngữ Dart. Tiến trình thực hiện Marshalling bằng cách chuyển đổi đối tượng này thành chuỗi JSON, sau đó mã hóa thành mảng byte theo chuẩn UTF-8 để gửi qua TCP Socket. Tại phía nhận, tiến trình thực hiện Unmarshalling bằng cách giải mã mảng byte thành chuỗi JSON và parse ngược lại thành đối tượng dữ liệu để hiển thị lên giao diện người dùng.

- Tính trong suốt: Nhờ sử dụng các thư viện phần mềm trung gian hỗ trợ đóng gói tự động, quá trình này diễn ra một cách trong suốt, giúp người lập trình không cần can thiệp sâu vào việc xử lý từng byte dữ liệu thô, từ đó giảm thiểu lỗi và tăng tốc độ phát triển.

Việc lựa chọn cơ chế đóng gói dựa trên JSON kết hợp với giao thức TCP đảm bảo rằng hệ thống PeerBuzz đạt được sự cân bằng giữa tính linh hoạt, khả năng mở rộng và độ tin cậy trong truyền thông phân tán.

2.4. Đồng hồ Logic Lamport

Trong một hệ thống phân tán như mạng chat ngang hàng PeerBuzz, việc duy trì một khái niệm thời gian thống nhất là một thách thức lớn. Do mỗi nút mạng chạy trên một phần cứng riêng biệt với bộ dao động tinh thể thạch anh có tần số khác nhau, các đồng hồ vật lý sẽ không tránh khỏi hiện tượng sai lệch. Trên mạng WAN việc đồng bộ thời gian tuyệt đối giữa các máy tính là cực kỳ khó khăn và tốn kém. Tuy nhiên, đối với nhiều ứng dụng phân tán, việc biết chính xác thời gian vật lý không quan trọng bằng việc xác định thứ tự xảy ra của các sự kiện. Leslie Lamport đã đề xuất khái niệm Đồng hồ Logic vào năm 1978 [2] để giải quyết vấn đề này mà không cần dựa vào thời gian vật lý hay độ trễ mạng.

Quan hệ Happens-before:

Nền tảng của đồng hồ Lamport là quan hệ nhân quả được ký hiệu là $a \rightarrow b$: sự kiện a xảy ra trước sự kiện b . Mỗi quan hệ này được xác định dựa trên hai tình huống trực tiếp:

- Nếu a và b là hai sự kiện trong cùng một tiến trình và a xuất hiện trước b , thì $a \rightarrow b$ là đúng.
- Nếu a là sự kiện gửi một thông điệp từ một tiến trình và b là sự kiện nhận thông điệp đó ở một tiến trình khác, thì $a \rightarrow b$ luôn đúng vì thông điệp không thể được nhận trước khi nó được gửi đi.

Quan hệ này có tính chất bắc cầu: nếu $a \rightarrow b$ và $b \rightarrow c$ thì $a \rightarrow c$. Trong trường hợp hai sự kiện x và y xảy ra ở hai tiến trình khác nhau mà không có thông điệp nào trao đổi trực tiếp hoặc gián tiếp giữa chúng, ta gọi đó là các sự kiện tương tranh - concurrent, nghĩa là không thể xác định quan hệ nhân quả giữa chúng.

Giải thuật gán nhãn thời gian Lamport:

Để hiện thực hóa việc theo dõi thứ tự, mỗi tiến trình P_i duy trì một bộ đếm cục bộ C_i đóng vai trò đồng hồ logic theo các quy tắc sau:

- Sự kiện nội tại: Trước khi thực hiện một sự kiện như gửi tin hoặc lưu dữ liệu, tiến trình tăng bộ đếm thêm một đơn vị: $C_i \leftarrow C_i + 1$

- **Gửi tin nhắn:** Khi tiến trình P_i gửi thông điệp m , nó đính kèm giá trị thời gian logic hiện tại vào thông điệp đó: $ts(m)=C_i$
- **Nhận tin nhắn:** Khi tiến trình P_j nhận được thông điệp m kèm nhãn $ts(m)$, nó cập nhật đồng hồ của mình bằng cách lấy giá trị lớn nhất giữa bộ đếm hiện tại và nhãn nhận được, sau đó tăng thêm 1 đơn vị: $C_j \leftarrow \max\{C_j, ts(m)\} + 1$
- Cơ chế này đảm bảo rằng nhãn thời gian của sự kiện nhận luôn lớn hơn nhãn thời gian của sự kiện gửi, giúp duy trì tính nhất quán nhân quả trên toàn hệ thống

Thứ tự toàn cục:

Một hạn chế của đồng hồ Lamport là có thể tồn tại hai sự kiện khác nhau có cùng một giá trị nhãn thời gian. Để tạo ra một thứ tự toàn cục duy nhất cho mọi sự kiện trong hệ thống, nhóm áp dụng phương pháp mở rộng nhãn thời gian bằng cách đính kèm định danh duy nhất của tiến trình: $\langle C_i, P_i \rangle$. Khi so sánh hai sự kiện $\langle C_a, P_a \rangle$ và $\langle C_b, P_b \rangle$:

- Sự kiện a được coi là xảy ra trước b nếu $C_a < C_b$
- Nếu $C_a = C_b$, sự kiện nào có định danh tiến trình nhỏ hơn sẽ được ưu tiên xếp trước

Ứng dụng trong hệ thống PeerBuzz:

Trong dự án PeerBuzz, đồng hồ logic Lamport đóng vai trò then chốt trong việc quản lý hội thoại nhóm. Khi nhiều người dùng trong một nhóm chat gửi tin nhắn gần như đồng thời, do độ trễ mạng khác nhau, các tin nhắn có thể đến các Peer theo thứ tự khác nhau. Nếu không có cơ chế đồng bộ, người dùng A có thể thấy tin nhắn của B trước tin nhắn của C, trong khi người dùng D lại thấy ngược lại, gây ra sự mất nhất quán trong trải nghiệm người dùng. Bằng cách gán nhãn thời gian Lamport cho mỗi tin nhắn, PeerBuzz đảm bảo:

1. **Tính nhất quán thứ tự:** Tất cả các Peer trong mạng đều sắp xếp và hiển thị dòng tin nhắn theo cùng một trình tự logic dựa trên nhãn thời gian đính kèm.
2. **Bảo toàn tính nhân quả:** Tin nhắn phản hồi sẽ luôn có nhãn thời gian lớn hơn tin nhắn gốc, đảm bảo nội dung hội thoại luôn mạch lạc ngay cả khi mạng có độ trễ lớn.
3. **Hỗ trợ chịu lỗi:** Khi một Peer bị ngắt kết nối và gia nhập lại, nó có thể sử dụng nhãn thời gian để yêu cầu đồng bộ các tin nhắn còn thiếu theo đúng thứ tự mà nó chưa nhận được.

Việc vận dụng đồng hồ Lamport giúp PeerBuzz đạt được tính trong suốt về phân phối, biến một mạng lưới các thiết bị rời rạc thành một không gian giao tiếp thống nhất và đáng tin cậy.

2.5. Tính chịu lỗi và bài toán Churn

Trong một hệ thống phân tán, một đặc điểm cốt lõi phân biệt với các hệ thống tập trung là khái niệm thất bại từng phần. Một phần của hệ thống có thể bị hỏng trong khi các phần còn lại vẫn tiếp tục vận hành bình thường. Để đảm bảo tính ổn định, hệ thống cần có khả năng chịu, nghĩa là có thể che giấu sự xuất hiện của lỗi hoặc tự phục hồi để tiếp tục cung cấp dịch vụ ở mức chấp nhận được.

Các mô hình thất bại trong hệ thống:

Dựa trên lý thuyết về hệ thống phân tán, nhóm xác định hai mô hình thất bại chính có thể xảy ra trong dự án chat ngang hàng:

- Thất bại sụp đổ - Crash failure: Một nút mạng đột ngột ngừng hoạt động mà không có thông báo trước. Trong PeerBuzz, điều này xảy ra khi ứng dụng bị tắt đột ngột hoặc thiết bị mất nguồn điện.
- Thất bại bỏ sót - Omission failure: Nút mạng không phản hồi các yêu cầu gửi đến hoặc không nhận được các thông điệp truyền tới. Điều này thường do lỗi kênh truyền thông hoặc tràn vùng đệm tại các Peer.

Bài toán nút gia nhập và rời mạng liên tục:

Churn là một đặc tính điển hình và thách thức nhất của các mạng ngang hàng. Hiện tượng này xảy ra khi các Peer tham gia và rời khỏi mạng một cách chủ động hoặc bị động với tần suất cao. Trong hệ thống chat P2P, Churn gây ra các vấn đề sau:

- Mất tính nhất quán danh sách online: Nếu một nút rời mạng mà không kịp thông báo, các Peer khác vẫn hiển thị trạng thái Online của nút đó, dẫn đến việc gửi tin nhắn bị thất bại liên tục.
- Đứt gãy mạng phủ: Trong các mạng P2P có cấu trúc, Churn có thể làm gián đoạn bảng định tuyến và các chuỗi liên kết giữa các nút.

Cơ chế chịu lỗi và quản lý Churn trong dự án:

Để đối phó với bài toán Churn và đảm bảo tính chịu lỗi, PeerBuzz áp dụng các kỹ thuật sau:

1. Phát hiện lỗi qua cơ chế Heartbeat: Nhóm sử dụng cơ chế Timeout kết hợp với thông điệp kiểm tra định kỳ. Các Peer sẽ định kỳ gửi tín hiệu Keep-alive đến Bootstrap Server hoặc các láng giềng. Nếu sau một khoảng thời gian t mà không nhận được tín hiệu, nút đó sẽ bị coi là đã sụp đổ và bị loại khỏi danh sách Active View
2. Dự thừa thông tin: Nhóm vận dụng phương pháp nhân bản trạng thái trên Bootstrap Server để lưu trữ danh sách các Peer đang trực tuyến. Khi một Peer mới gia nhập,

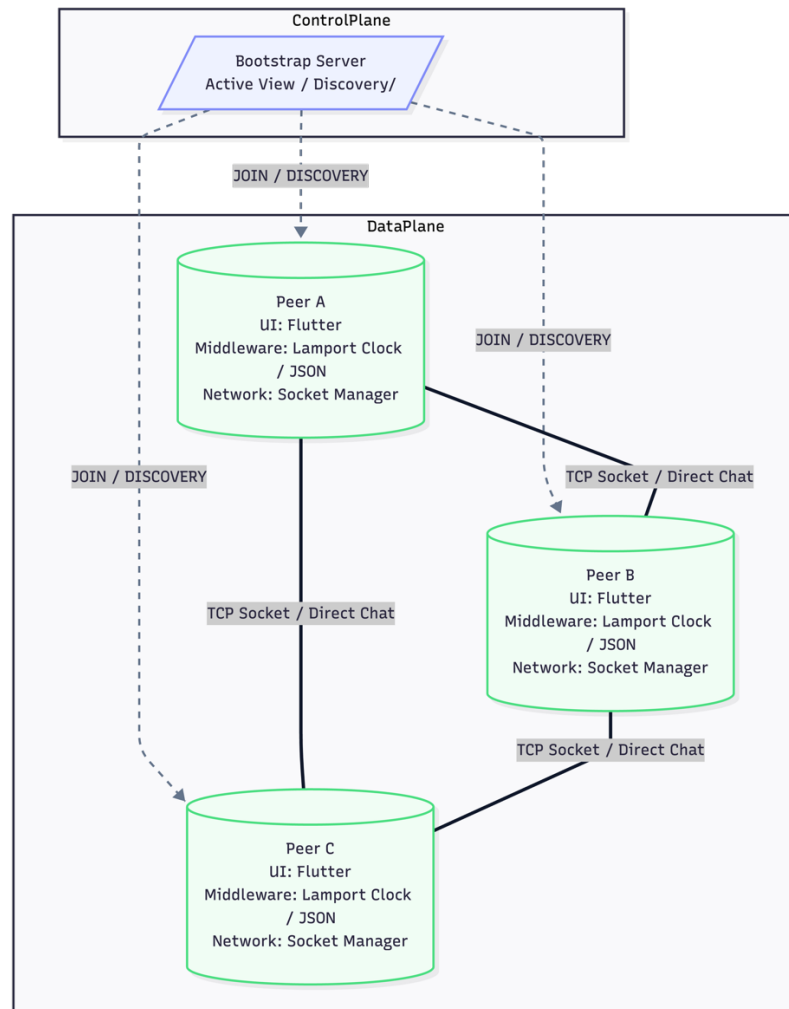
máy chủ này cung cấp danh sách láng giềng dự phòng để đảm bảo Peer luôn có thể thiết lập ít nhất một kết nối mạng phủ

3. Tự phục hồi và Kết nối lại: Khi một Peer phát hiện mất kết nối TCP Socket tới một người bạn, nó sẽ tự động thử thiết lập lại kết nối hoặc truy vấn Bootstrap Server để lấy địa chỉ IP mới, đảm bảo tính trong suốt về lỗi đối với người dùng
4. Cập nhật trạng thái thời gian thực: Vận dụng mô hình Hybrid P2P, Bootstrap Server giữ vai trò điều phối tập trung danh sách online để xử lý nhanh chóng các yêu cầu đăng ký và rời mạng, giúp giảm thiểu thời gian trễ khi cập nhật hình trạng mạng.

Tóm lại, việc hiểu rõ các mô hình thất bại và áp dụng các cơ chế phát hiện lỗi chủ động giúp hệ thống PeerBuzz duy trì được sự ổn định ngay cả trong môi trường mạng P2P có độ biến động cao, đáp ứng các mục tiêu về tính sẵn sàng và tin cậy của môn học.

Chương 3: Xây dựng hệ thống

3.1. Kiến trúc hệ thống



Hình 1: Sơ đồ kiến trúc hệ thống

Hệ thống PeerBuzz được thiết kế dựa trên mô hình kiến trúc Hybrid P2P. Đây là sự lựa chọn tối ưu nhằm kết hợp ưu điểm của tính quản lý tập trung trong việc tìm kiếm thực thể và tính mạnh mẽ, hiệu năng của mạng phi tập trung trong truyền tải dữ liệu. Cấu trúc này chia hệ thống thành hai thành phần chính với các vai trò logic riêng biệt:

- **Bootstrap Server:** Đóng vai trò là một thực thể trung tâm đơn giản nhưng cực kỳ quan trọng để khởi tạo mạng lưới.
 - **Quản lý danh sách Peer đang hoạt động (Active View):** Máy chủ này lưu giữ một bảng bản ghi bao gồm định danh Peer và địa chỉ mạng tương ứng.
 - **Hỗ trợ khám phá (Discovery Helper):** Khi một Peer mới gia nhập mạng, nó sẽ liên lạc với Bootstrap Server để đăng ký thông tin cá nhân và nhận về danh sách các Peer khác đang trực tuyến.

- Giảm thiểu rào cản mạng: Bootstrap Server giúp giải quyết vấn đề của các nút nằm sau tường lửa hoặc sử dụng IP động bằng cách cung cấp một rendezvous point cố định.
- **Peer Node:** Mỗi nút mạng trong PeerBuzz vận hành theo mô hình Servant. Đây là đặc tính cốt lõi của các hệ thống P2P hiện đại:
 - Vai trò máy chủ: Peer thực hiện mở một cổng TCP Socket để lắng nghe và chấp nhận các yêu cầu kết nối từ các láng giềng khác. Điều này cho phép Peer trực tiếp nhận tin nhắn chat mà không cần thông qua máy chủ trung gian.
 - Vai trò máy khách: Peer chủ động thiết lập kết nối tới địa chỉ IP của các Peer khác dựa trên thông tin nhận được từ Bootstrap Server để gửi tin nhắn.
 - Tính tự trị: Mỗi Peer là một đơn vị tính toán tự trị, có khả năng xử lý đồng thời nhiều phiên làm việc để gửi và nhận tin cùng một lúc.

Kiến trúc PeerBuzz hình thành một mạng phủ logic bên trên hạ tầng Internet vật lý.

- Liên kết mạng: Các mối quan hệ láng giềng giữa các Peer tạo thành một đồ thị kết nối. Trong PeerBuzz, nhóm sử dụng cấu trúc mạng phủ phi cấu trúc cho phép các nút duy trì danh sách hàng xóm một cách linh hoạt, phù hợp với các hệ thống có tính động cao.
- Tính trong suốt về vị trí: Thông qua sự phối hợp giữa Bootstrap Server và mạng phủ, người dùng cuối có thể tương tác với bạn bè bằng tên hoặc định danh mà không cần quan tâm đến sự phức tạp của việc phân giải địa chỉ IP hay quản lý các kết nối Socket bên dưới.

Sự kết hợp giữa Bootstrap Server và các nút Servant tạo nên một hệ thống bền bỉ: luồng dữ liệu chat được phân tán hoàn toàn giúp loại bỏ nguy cơ nút cổ chai tại máy chủ, đồng thời đảm bảo tính sẵn sàng cao ngay cả khi một phần của mạng lưới gặp sự cố.

3.2. Giao thức trao đổi thông điệp

Nền tảng truyền thông Socket TCP: Hệ thống PeerBuzz sử dụng giao thức TCP tại tầng vận tải làm nền tảng cho việc trao đổi dữ liệu trực tiếp giữa các Peer. Lựa chọn này xuất phát từ yêu cầu khắt khe của một ứng dụng chat về tính tin cậy:

- Thiết lập liên kết: Trước khi trao đổi tin nhắn, hai Peer thực hiện thủ tục bắt tay để thiết lập một kênh truyền hướng kết nối.

Bảng 1: Đoạn mã thiết lập kết nối TCP từ Peer

```
final socket = await Socket.connect(
```

```

        target.ip,
        target.port,
        timeout: const Duration(seconds: 5),
    );

```

- Bảo toàn thứ tự: TCP đảm bảo các gói tin đến đích theo đúng trình tự đã gửi, điều này cực kỳ quan trọng để duy trì mạch hội thoại logic.
- Kiểm soát lỗi: Cơ chế xác nhận và truyền lại của TCP giúp hệ thống tự động khắc phục các lỗi mất gói tin trên mạng mà không cần can thiệp ở tầng ứng dụng.

Mỗi Peer hoạt động như một Servant, thực hiện đồng thời việc lắng nghe trên một cổng TCP cố định và chủ động kết nối tới cổng của Peer khác bằng các hàm nguyên thủy như *socket()*, *listen()*, *accept()* và *connect()*

Định dạng và Đóng gói thông điệp: Để các nút mạng có thể giải mã chính xác thông tin, PeerBuzz chuẩn hóa cấu trúc dữ liệu qua định dạng JSON. Quá trình Marshalling chuyển đổi các đối tượng tin nhắn thành chuỗi byte để truyền qua Socket và Unmarshalling tại phía nhận để tái tạo lại dữ liệu ban đầu. Một thông điệp chuẩn trong hệ thống bao gồm các trường thông tin cốt lõi:

- Type: Phân loại thông điệp là CHAT_MSG - tin nhắn văn bản, CONTROL_JOIN - thông báo gia nhập, hoặc HEARTBEAT - duy trì kết nối

Bảng 2: Định nghĩa các loại thông điệp trong giao thức

```

enum MessageType {
    join, // Peer -> Bootstrap: Tham gia mạng
    leave, // Peer -> Bootstrap: Rời mạng chủ động
    heartbeat, // Peer -> Bootstrap: Duy trì online
    chatReq, // Peer A -> Peer B: Yêu cầu chat
    chatMsg, // Peer -> Peer: Gửi tin nhắn
    broadcastMsg, // Peer -> All: Chat nhóm
    fileReq, // Yêu cầu gửi file
    fileChunk, // Khối dữ liệu file
    storeFor, // Peer A -> Peer C (relay): Nhờ C lưu hộ tin khi B offline; khi B
    online, C chủ động giao tin
    gossipDown, // Peer -> Peer: Lan truyền nút bị sập
    updateList // Bootstrap -> Peer: Cập nhật danh sách online
}

```

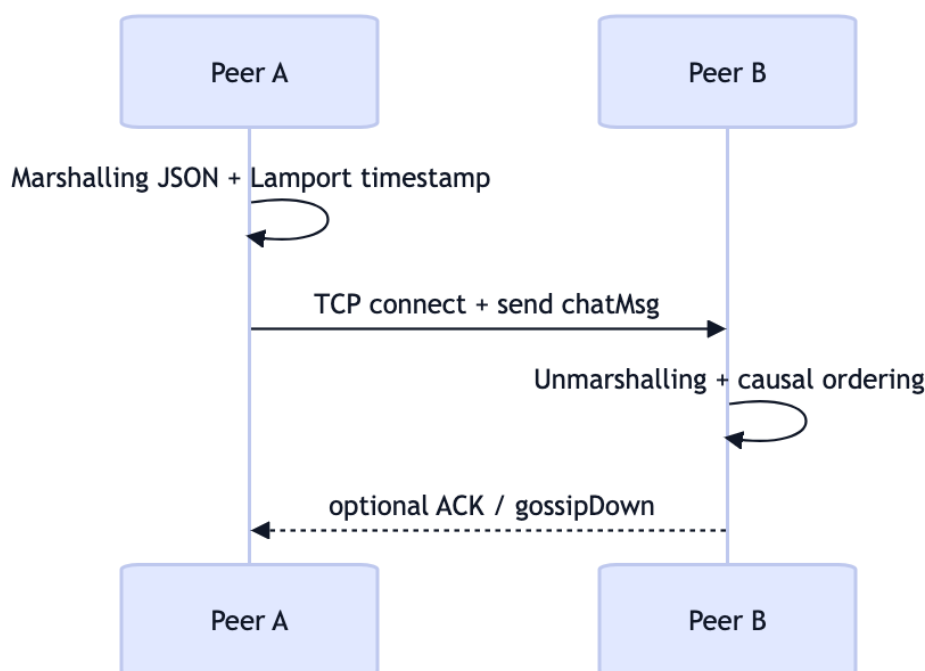
- SenderID & ReceiverID: Định danh duy nhất của các Peer tham gia giao tiếp
- Payload: Chứa nội dung văn bản thực tế của tin nhắn hoặc các tệp tin đã được mã hóa
- Lamport Timestamp: Nhãn thời gian logic dùng để định danh thứ tự toàn cục của tin nhắn

Cơ chế store-and-forward: Khi Peer A không thể kết nối trực tiếp đến Peer B (do B ngoại tuyến), A chọn ngẫu nhiên một láng giềng C còn trực tuyến và gửi thông điệp storeFor kèm định danh của B và nội dung tin nhắn gốc. C lưu giữ tin nhắn này trong bộ nhớ. Khi B kết nối trở lại và Bootstrap Server phát tán danh sách mới, C nhận biết B đã online và chủ động giao tin tồn đọng. Hàng đợi lưu trữ tại mỗi nút relay được giới hạn tối đa 100 tin nhắn để kiểm soát bộ nhớ.

Bảng 3: Cấu trúc thông điệp chat có gắn nhãn Lamport

```
final msg = BaseMessage(
    type: MessageType.chatMsg,
    senderId: localPeer.id,
    senderName: localPeer.name,
    lamportTimestamp: clock.tick(),
    payload: {
        'content': await EncryptionHelper.encrypt(content),
        'is_encrypted': true,
    },
);
```

3.2.1. Quy trình trao đổi tin nhắn trực tiếp 1 - 1



Hình 2: Luồng giao tiếp giữa hai Peer chat 1 - 1

Quy trình trao đổi giữa hai Peer A và B diễn ra qua các bước sau:

- 1. Phát hiện và Kết nối:** Peer A lấy địa chỉ IP và Port của Peer B từ danh sách láng giềng được Bootstrap Server cung cấp. Sau đó, A thực hiện lệnh *connect()* tới B

2. **Gửi thông điệp:** Khi người dùng A nhấn gửi, hệ thống tăng đồng hồ logic Lamport cục bộ ($C_A \leftarrow C_A + 1$), đóng gói tin nhắn kèm nhãn thời gian và đẩy vào vùng đệm của Socket
3. **Tiếp nhận và Xử lý:** Peer B nhận chuỗi byte từ hàm *recv()*, giải gói JSON và cập nhật đồng hồ logic của mình theo quy tắc $C_B \leftarrow \max\{C_B, ts(m)\} + 1$. Điều này đảm bảo rằng tin nhắn nhận được luôn có nhãn thời gian logic lớn hơn thời điểm nó được gửi đi.

3.2.2. Quy trình trao đổi tin nhắn nhóm

Đối với tính năng chat nhóm, PeerBuzz áp dụng cơ chế Truyền thông đa hướng thứ tự toàn cục, Total-ordered Multicasting dựa trên nhãn thời gian Lamport.

- Cơ chế lan tỏa: Khi một Peer gửi tin nhắn vào nhóm, thông điệp sẽ được multicast tới tất cả các thành viên trong nhóm mạng phủ
- Bộ đệm tin nhắn: Tại mỗi Peer nhận, các tin nhắn nhóm được tích lũy vào một danh sách tạm trong bộ nhớ ứng dụng ngay khi đến.
- Sắp xếp và hiển thị: Trước khi hiển thị lên giao diện người dùng, danh sách tin nhắn được sắp xếp tăng dần theo nhãn thời gian Lamport. Cơ chế này đảm bảo rằng dù các tin nhắn đến tại mỗi Peer theo thứ tự khác nhau do độ trễ mạng, kết quả hiển thị cuối cùng trên tất cả các thiết bị đều phản ánh cùng một thứ tự nhân quả nhất quán. Đây là cách triển khai đơn giản và hiệu quả của nguyên lý Total-ordered Multicasting trong môi trường mạng P2P có quy mô nhỏ.

3.2.3. Thông điệp điều khiển và xử lý rời mạng

Giao thức của PeerBuzz còn bao gồm các thông điệp điều khiển để quản lý trạng thái động của hệ thống phân tán:

- JOIN/LEAVE: Khi một Peer gia nhập hoặc rời mạng một cách chủ động, nó gửi thông điệp thông báo để các láng giềng cập nhật danh sách hiển thị
- Keep-alive/Heartbeat: Để xử lý lỗi sập đổ, các Peer định kỳ trao đổi các gói tin kiểm tra trạng thái cục bộ. Nếu một Peer không phản hồi sau một khoảng thời gian nhất định, giao thức sẽ coi nút đó đã rời mạng và ngắt kết nối Socket tương ứng để giải phóng tài nguyên

Tóm lại, giao thức trao đổi thông điệp của PeerBuzz không chỉ đơn thuần là việc truyền dữ liệu văn bản, mà là sự phối hợp chặt chẽ giữa các giải thuật định dạng dữ liệu JSON, truyền thông tin cậy TCP và điều phối thời gian logic Lamport. Sự kết hợp này giúp hệ

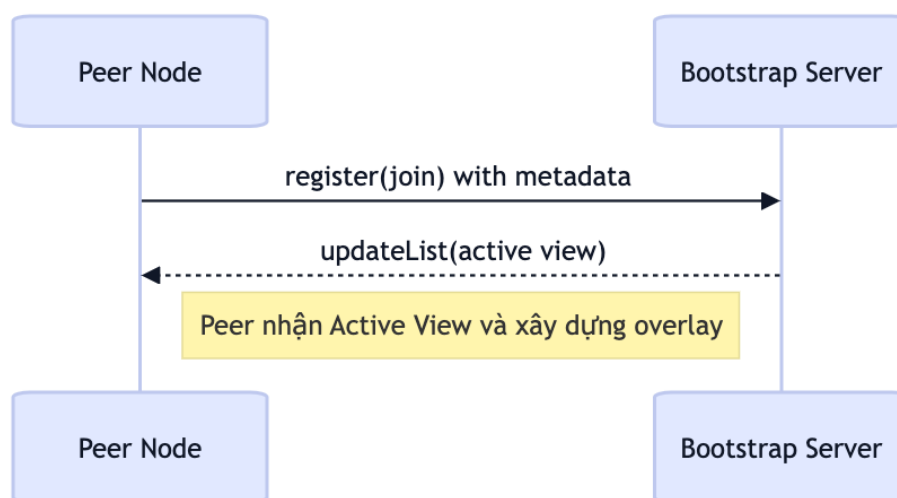
thống đạt được tính trong suốt về phân phối, cho phép người dùng giao tiếp một cách liên mạch bất kể sự phức tạp của hạ tầng mạng bên dưới.

3.3. Cơ chế Peer Discovery

Bài toán đầu tiên và quan trọng nhất cần giải quyết là làm thế nào để một tiến trình mới gia nhập có thể tìm thấy và thiết lập liên kết với các thành viên khác. Quá trình này được gọi là Peer Discovery (khám phá nút). Đối với hệ thống PeerBuzz, việc thiết kế cơ chế này không chỉ đơn thuần là tìm kiếm địa chỉ IP, mà còn là bước khởi tạo để xây dựng một mạng phủ (overlay network) logic ổn định bên trên hạ tầng Internet vật lý.

Việc khám phá nút trong môi trường mạng WAN đối mặt với nhiều trở ngại:

- Tính động: Các nút tham gia và rời mạng liên tục mà không báo trước, làm cho danh sách địa chỉ IP nhanh chóng bị lỗi thời
- Địa chỉ không cố định: Hầu hết người dùng cá nhân sử dụng IP động và nằm sau các lớp tường lửa hoặc thiết bị NAT, khiến việc kết nối trực tiếp trở nên khó khăn nếu không có thực thể hỗ trợ
- Hạn chế của Broadcasting: Trong mạng LAN, ta có thể sử dụng cơ chế Broadcasting để tìm thực thể. Tuy nhiên, trên Internet, các thiết bị định tuyến sẽ chặn các gói tin quảng bá này để tránh nghẽn mạng, do đó giải pháp này không khả thi cho một ứng dụng chat quy mô lớn.



Hình 3: Luồng giao tiếp giữa Peer và Bootstrap Server trong pha Discovery

Để vượt qua các rào cản trên, PeerBuzz vận dụng mô hình kiến trúc Hybrid P2P, sử dụng một Bootstrap Server làm điểm hẹn cố định cho toàn hệ thống. Cơ chế khám phá diễn ra theo quy trình kỹ thuật gồm 4 bước chính:

- Bước 1: Khởi tạo và Đăng ký: Khi một Peer Node khởi động, nó sẽ nạp địa chỉ IP và Port của Bootstrap Server từ cấu hình hệ thống. Peer gửi một thông điệp JOIN chứa định danh PeerID duy nhất và thông tin điểm truy nhập hiện tại của nó tới Bootstrap Server qua Socket TCP
- Bước 2: Quản lý Danh sách hoạt động: Bootstrap Server tiếp nhận yêu cầu, cập nhật thông tin của Peer mới vào cơ sở dữ liệu tạm thời. Máy chủ này đóng vai trò như một Tracker, duy trì một cái nhìn chính xác về các nút đang thực sự trực tuyến trong mạng
- Bước 3: Phản hồi danh sách láng giềng: Sau khi đăng ký thành công, Bootstrap Server phản hồi cho Peer mới một danh sách chứa thông tin của một tập hợp các Peer khác đang hoạt động. Việc gửi một tập con thay vì toàn bộ danh sách giúp hệ thống giảm tải truyền thông và tăng tính bảo mật
- Bước 4: Thiết lập Mạng phủ: Nhận được danh sách từ Bootstrap Server, Peer mới sẽ chọn ra các Peer mục tiêu và chủ động thực hiện lệnh *connect()* để thiết lập các kênh truyền TCP Socket trực tiếp với các láng giềng này. Tại thời điểm này, Peer chính thức trở thành một phần của mạng phủ phi cấu trúc và có thể bắt đầu trao đổi tin nhắn chat mà không cần sự can thiệp thêm của Bootstrap Server

Cơ chế khám phá không chỉ diễn ra một lần mà là một quá trình liên tục để đối phó với hiện tượng Churn. PeerBuzz kết hợp cơ chế timeout tại Bootstrap Server với kỹ thuật lan truyền lỗi dựa trên tin đồn (gossip-based failure detection) [6] để duy trì tính chính xác của thông tin láng giềng:

- Phát hiện và lan truyền trạng thái lỗi: Khi một kết nối TCP tới một láng giềng bị ngắt hoặc không phản hồi, Peer phát hiện lỗi sẽ phát tán thông điệp gossipDown tới một tập nhỏ các láng giềng còn lại. Cơ chế này nhanh chóng hội tụ thông tin về trạng thái nút chết tới toàn bộ mạng phủ mà không cần Bootstrap Server phải chủ động thông báo. Trong phiên bản hiện tại, gossip được sử dụng chuyên biệt cho phát hiện lỗi (failure notification), là một ứng dụng quan trọng của nguyên lý Gossip-based failure detection [6]. Việc mở rộng sang Gossip-based Peer Sampling đầy đủ, trong đó các Peer trao đổi toàn bộ danh sách láng giềng để duy trì tính tươi mới của partial view được xác định là hướng phát triển tiếp theo của hệ thống.
- Phát hiện nút chết: Nếu một kết nối Socket tới láng giềng bị ngắt hoặc không phản hồi sau một khoảng thời gian, Peer sẽ loại bỏ nút đó khỏi danh sách cục bộ và yêu cầu Bootstrap Server cung cấp các láng giềng mới để thay thế

Bảng 4: Xử lý các loại thông điệp tại Bootstrap Server

```
void _processMessage(Map<String, dynamic> msg, Socket client) {
    final type = msg['type'];
    final senderId = msg['sender_id'];
    final payload = msg['payload'] ?? {};

    switch (type) {
        case 'join':
            print(
                '(+) Peer JOIN: ${payload['name']} ($senderId) at
                ${client.remoteAddress.address}');
            _updatePeer(senderId, payload, client.remoteAddress.address);
            _sendPeerList(client);
            break;

        case 'heartbeat':
            // Cập nhật trạng thái sống mà không cần in log quá nhiều
            _updatePeer(senderId, payload, client.remoteAddress.address);
            _sendPeerList(client); // Trả về danh sách mới nhất để Peer cập nhật
            break;

        case 'leave':
            print('(-) Peer LEAVE: $senderId');
            _activePeers.remove(senderId);
            break;

        case 'gossipDown':
            final targetId = payload['target_id'];
            if (_activePeers.containsKey(targetId)) {
                print(
                    '(!) Gossip received: Node $targetId might be down. Verifying...');
                // Trong mô hình tin cậy, ta có thể xóa luôn hoặc đợi Timeout
                // ở đây ta xóa luôn để tăng tốc độ hội tụ mạng
                _activePeers.remove(targetId);
            }
            break;
    }
}
```

3.4. Xử lý lỗi và thử nghiệm hệ thống

3.4.1. Các mô hình lỗi và giải pháp xử lý

Mô hình 1: Crash Failure - Thất bại sụp đổ

- Xảy ra khi một Peer hoặc Bootstrap Server đột ngột ngừng hoạt động do lỗi phần cứng, mất điện hoặc người dùng tắt ứng dụng đột ngột
- Giải pháp xử lý: Hệ thống áp dụng cơ chế Phát hiện lỗi dựa trên thời gian quá hạn. Mỗi Peer duy trì kết nối Heartbeat định kỳ tới Bootstrap Server. Nếu sau một khoảng thời gian cấu hình mà Bootstrap Server không nhận được tín hiệu Keep-alive, nút đó sẽ bị loại khỏi danh sách Active View. Đối với các phiên chat trực tiếp, nếu một

Peer phát hiện Socket bị ngắt kết nối đột ngột, nó sẽ thông báo cho người dùng và cố gắng thiết lập lại liên kết sau một khoảng trễ.

Mô hình 2: Omission Failure - Thất bại bỏ sót

- Xảy ra khi tin nhắn bị thất lạc trên đường truyền mạng
- Giải pháp xử lý: Bằng việc sử dụng giao thức TCP Socket làm nền tảng truyền thông, PeerBuzz tận dụng khả năng tự động xác nhận và truyền lại của tầng vận tải để che giấu các lỗi bỏ sót này một cách trong suốt đối với tầng ứng dụng.

3.4.2. Thử nghiệm hệ thống

Môi trường thử nghiệm:

Bảng 5: Môi trường thử nghiệm hệ thống

	Môi trường thử nghiệm
Cấu hình	Triển khai Bootstrap Server trên một máy tính và chạy nhiều Peer trên các thiết bị khác nhau: laptop Windows, Mac, Web, Smartphone Android, smartphone iOS trên cùng một mạng Wi-Fi cục bộ
Mục tiêu	Kiểm tra khả năng Peer Discovery, tốc độ truyền tin nhắn và tính chính xác của giao diện hiển thị trạng thái Online/Offline khi các Peer Join và Leave mạng

Kịch bản thử nghiệm:

Bảng 6: Các kịch bản thử nghiệm và kết quả mong đợi

TT	Kịch bản thử nghiệm	Kết quả mong đợi	Lý thuyết áp dụng
1	Peer Join mạng	Đăng ký thành công với Bootstrap Server và nhận được danh sách Peer khác	Peer Discovery
2	Chat nhóm	Tất cả các Peer nhận được tin nhắn và hiển thị theo đúng thứ tự gửi	Total-ordered Multicasting
3	Chat 1 - 1	2 Peer gửi và nhận tin nhắn tức thì, tin nhắn hiển thị theo đúng thứ tự gửi	TCP Socket
4	Peer Leave mạng đột ngột	Bootstrap Server cập nhật danh sách sau t giây timeout	Crash Failure Detection

5	Chat khi mạng có độ trễ lớn	Tin nhắn đến sau vẫn được sắp xếp đúng vị trí nhân quả nhờ nhãn thời gian	Lamport Clock
6	Mô phỏng Churn: nhiều Peer join và leave liên tục trong thời gian ngắn	Hệ thống hội tụ, danh sách online cập nhật chính xác sau mỗi chu kỳ timeout; các phiên chat đang diễn ra không bị gián đoạn	Churn Resilience, Heartbeat + Gossip

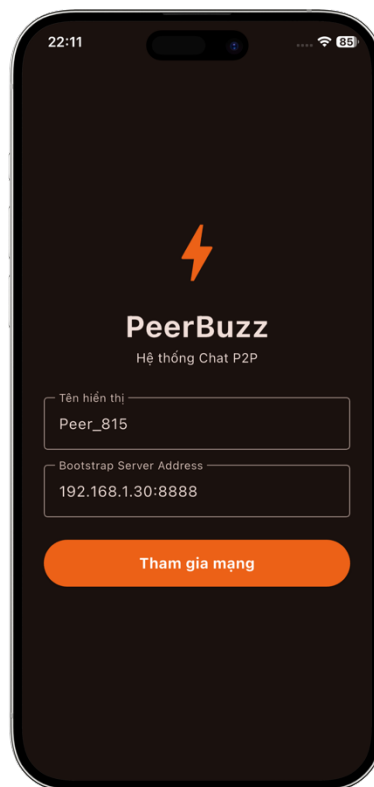
Chương 4: Kết quả và Đánh giá

Chương này trình bày toàn bộ kết quả thực nghiệm của đề tài, bao gồm việc giới thiệu giao diện các màn hình của Peer và đánh giá kết quả trên từng kịch bản thử nghiệm, sau đó đưa ra nhận định về kết quả đạt được và hạn chế của hệ thống.

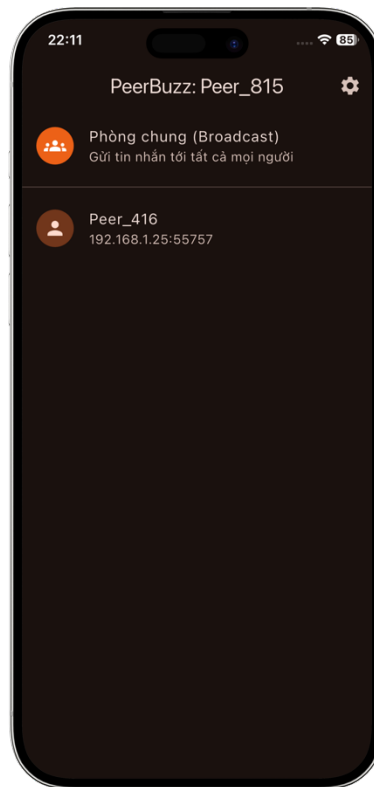
4.1. Giao diện ứng dụng

Phần này giới thiệu các màn hình giao diện chính của ứng dụng Peer được phát triển trên nền Flutter, cho phép triển khai đa nền tảng (Windows, macOS, Web, Android, iOS) với cùng một mã nguồn. Giao diện được thiết kế tối giản, tập trung vào các chức năng cốt lõi:

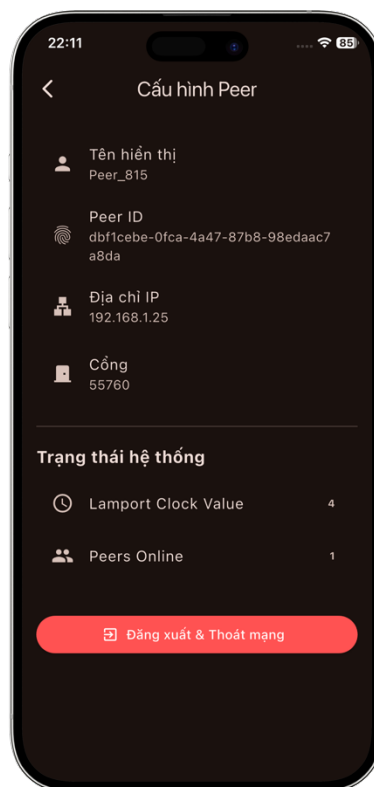
- Trao đổi tin nhắn 1 - 1



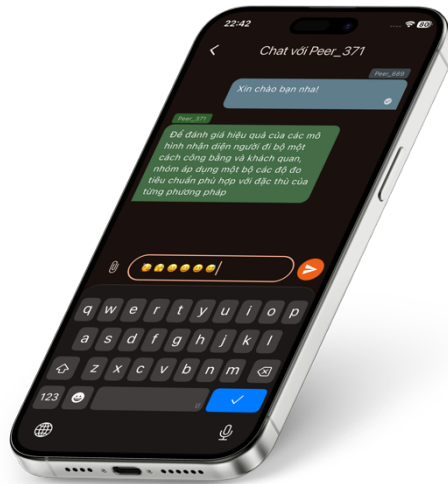
Hình 4: Màn hình đăng nhập Peer trên thiết bị di động



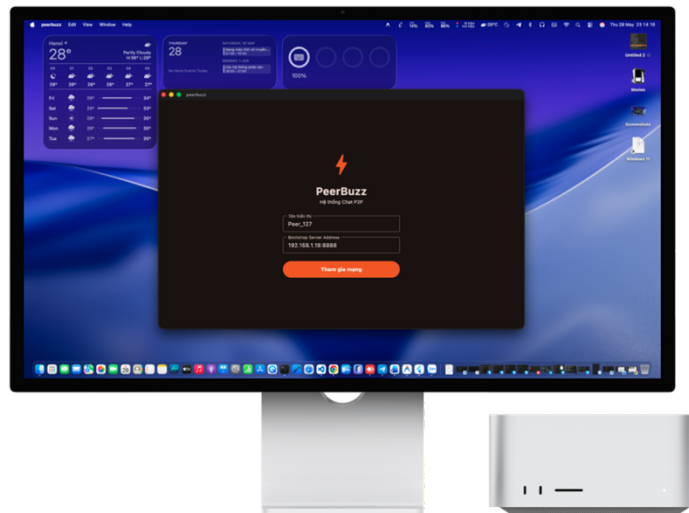
Hình 5: Danh sách Peer trực tuyến hiển thị trên thiết bị di động



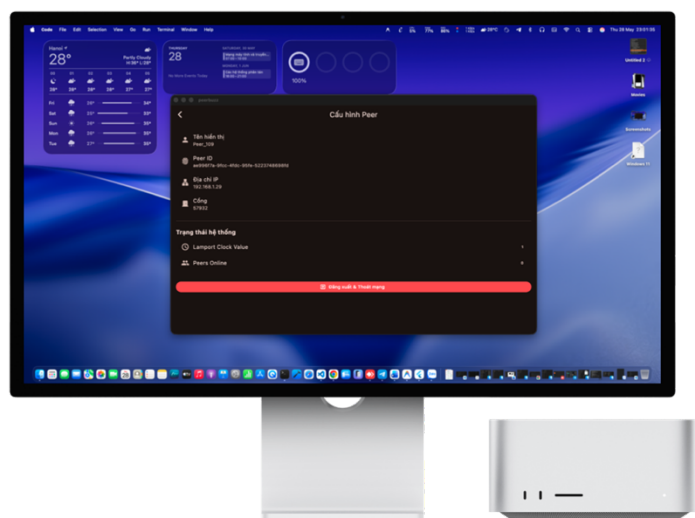
Hình 6: Giao diện thông tin Peer trên thiết bị di động



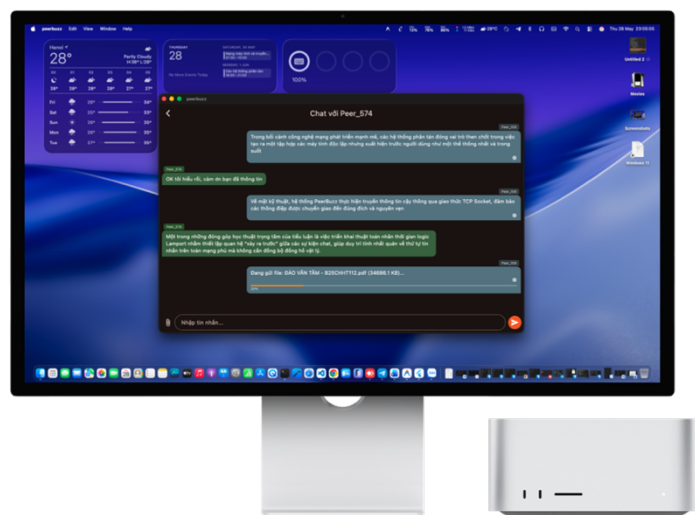
Hình 7: Giao diện chat 1 - 1 trên thiết bị di động



Hình 8: Màn hình đăng nhập Peer trên nền tảng Desktop



Hình 9: Màn hình thông tin Peer trên nền tảng Desktop



Hình 10: Màn hình chat 1 - 1 trên Desktop cùng tin nhắn gửi File



Hình 11: Màn hình chat 1 - 1 trên nền tảng Desktop

4.2. Các kịch bản thử nghiệm

Năm kịch bản thử nghiệm đã được thiết kế ở Mục 3.4.2 được tiến hành chạy thực tế trên môi trường LAN (Wi-Fi nội bộ). Mỗi kịch bản được thực hiện lặp lại nhiều lần với các thiết bị khác nhau nhằm xác minh tính ổn định và đo độ trễ truyền tin. Kết quả thu được được tổng hợp trong Bảng dưới đây, đối chiếu trực tiếp với kết quả mong đợi đã đề ra.

Bảng 7: Kết quả thực nghiệm các kịch bản

TT	Kịch bản	Kết quả thực tế	Đánh giá
1	Peer Join mạng	Peer mới đăng ký thành công với Bootstrap Server trong < 1 giây; nhận đầy đủ danh sách các Peer đang trực tuyến; các Peer khác cập nhật danh sách hiển thị ngay khi Peer mới tham gia	Đạt
2	Chat nhóm	Tin nhắn được multicast tới tất cả Peer trong nhóm; thứ tự hiển thị tin nhắn nhất quán trên mọi máy nhờ Lamport Clock, kể cả khi các Peer gửi gần như đồng thời	Đạt

3	<i>Chat 1 - 1</i>	<i>Hai Peer trao đổi tin nhắn qua kênh TCP trực tiếp với độ trễ trung bình < 200 ms; không có trường hợp mất tin nhắn</i>	<i>Đạt</i>
4	<i>Peer Leave mạng đột ngột</i>	<i>Sau khoảng 5 - 10 giây timeout, Bootstrap Server phát hiện Peer ngắt kết nối và phát tán danh sách Peer cập nhật; các Peer còn lại đánh dấu trạng thái Offline tương ứng</i>	<i>Đạt</i>
5	<i>Chat khi mạng có độ trễ lớn</i>	<i>Khi mô phỏng độ trễ mạng nhân tạo, các tin nhắn đến không theo thứ tự thời gian thực vẫn được sắp xếp đúng quan hệ nhân quả dựa trên nhãn Lamport, hiển thị nhất quán giữa các Peer</i>	<i>Đạt</i>
6	<i>Mô phỏng Churn</i>	<i>Với 4–6 Peer luân phiên join/leave mỗi 15 giây, Bootstrap Server hội tụ danh sách online chính xác sau mỗi chu kỳ timeout 60 giây; Gossip Down rút ngắn thời gian phát hiện nút lỗi xuống còn 5–10 giây; các phiên chat 1-1 đang diễn ra giữa các Peer ổn định không bị gián đoạn</i>	<i>Đạt</i>

4.3. Phân tích kết quả và hạn chế

Toàn bộ năm kịch bản thử nghiệm đều đạt kết quả mong đợi, cho thấy hệ thống đã đáp ứng các mục tiêu đề ra tại Chương 1 trên cả hai khía cạnh chức năng và lý thuyết hệ phân tán.

Kết quả đạt được:

1. Cơ chế Peer Discovery hoạt động ổn định: Bootstrap Server đăng ký, theo dõi và phát tán danh sách Peer kịp thời, cho phép một Peer mới tham gia mạng và biết về các Peer khác mà không cần cấu hình thủ công địa chỉ IP.
2. Kênh truyền tin TCP cho phép trao đổi tin nhắn 1 - 1 với độ trễ thấp, đảm bảo thứ tự FIFO trong từng kết nối điểm - điểm theo đặc tả của giao thức [3].

3. Việc áp dụng Đồng hồ Lamport [2] vào tin nhắn nhóm cho phép sắp xếp các sự kiện theo quan hệ nhân quả, giúp các Peer hiển thị thứ tự tin nhắn nhất quán ngay cả khi mạng có độ trễ lớn hoặc các Peer gửi đồng thời - đây là minh chứng thực nghiệm cho lý thuyết Total-ordered Multicasting đã trình bày ở Chương 2.
4. Cơ chế timeout của Bootstrap Server giúp phát hiện crash failure khi một Peer rời mạng đột ngột, bảo đảm danh sách Peer hiển thị trên các nút còn lại phản ánh đúng trạng thái thực tế.
5. Ứng dụng được triển khai bằng Flutter [7] cho phép chạy đồng nhất trên năm nền tảng (Windows, macOS, Web, Android, iOS) với cùng một bộ mã nguồn, xác nhận tính khả chuyển của thiết kế giao thức ở tầng ứng dụng.
6. Cơ chế lưu trữ lịch sử hội thoại cục bộ (StorageService) cho phép mỗi Peer tự quản lý và khôi phục toàn bộ lịch sử chat sau khi khởi động lại ứng dụng, nâng cao trải nghiệm người dùng mà không phụ thuộc vào máy chủ tập trung.
7. Cơ chế loại trùng thông điệp (Message Deduplication) thông qua tập hợp định danh tin nhắn đã xử lý (_seenMessageIds) đảm bảo mỗi tin nhắn broadcast chỉ được xử lý và hiển thị đúng một lần, ngay cả khi mạng phủ truyền dẫn nhiều bản sao do cấu trúc đồ thị kết nối.

Hạn chế của hệ thống:

1. Bootstrap Server vẫn đóng vai trò một thành phần tập trung. Mặc dù các Peer trao đổi tin nhắn trực tiếp với nhau theo mô hình ngang hàng, nếu Bootstrap Server gặp sự cố thì các Peer mới sẽ không thể tham gia mạng. Đây là single point of failure cần được khắc phục bằng cách triển khai nhiều Bootstrap Server theo cơ chế Gossip-based Peer Sampling [6] hoặc một bảng băm phân tán (DHT).
2. Phạm vi thử nghiệm còn nhỏ; chưa đánh giá được hiệu năng và tính mở rộng khi số lượng Peer lên đến hàng chục hoặc hàng trăm nút, đặc biệt với cơ chế multicast nguyên thủy hiện đang sử dụng vòng lặp gửi tuần tự tới từng Peer.
3. Hệ thống chưa hỗ trợ NAT traversal đầy đủ; trong trường hợp triển khai thực tế qua Internet, các Peer nằm sau NAT sẽ cần cấu hình port forwarding thủ công, thay vì được hỗ trợ tự động bởi các kỹ thuật STUN/TURN/ICE thường thấy ở các ứng dụng P2P thương mại. Đây là hướng phát triển tiếp theo của hệ thống.
4. Lịch sử hội thoại 1-1 được lưu cục bộ trên thiết bị của mỗi Peer dưới dạng tệp JSON thông qua StorageService, cho phép khôi phục khi khởi động lại ứng dụng. Tuy nhiên, tin nhắn được giao thông qua cơ chế store-and-forward chỉ tồn tại trong bộ

nhớ RAM của nút relay trung gian: nếu nút relay ngắt kết nối trước khi giao tin đến đích, các tin nhắn này sẽ bị mất vĩnh viễn. Để khắc phục triệt để, cần bổ sung cơ chế lưu trữ bền vững (persistent storage) cho hàng đợi store-and-forward tại tầng relay.

TÀI LIỆU THAM KHẢO

- [1] Tanenbaum, A. S., & Van Steen, M. (2017). Distributed Systems: Principles and Paradigms. 3rd ed., CreateSpace Independent Publishing Platform
- [2] Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM, 21(7), pp. 558-565
- [3] Postel, J. (1981). Transmission Control Protocol. RFC 793, Internet Engineering Task Force (IETF). Available tại: <https://www.rfc-editor.org/rfc/rfc793>
- [4] Stevens, W. R., Fenner, B., & Rudoff, A. M. (2003). UNIX Network Programming, Volume 1: The Sockets Networking API. 3rd ed., Addison-Wesley
- [5] Bray, T. (Ed.) (2017). The JavaScript Object Notation (JSON) Data Interchange Format. RFC 8259, Internet Engineering Task Force (IETF). Available tại: <https://www.rfc-editor.org/rfc/rfc8259>
- [6] Jelasity, M., Voulgaris, S., Guerraoui, R., Kermarrec, A.-M., & Van Steen, M. (2007). Gossip-based Peer Sampling. ACM Transactions on Computer Systems, 25(3), Article 8
- [7] Flutter (Google). (2024). Flutter - Build apps for any screen. Available tại: <https://flutter.dev>

----- *Hết tài liệu* -----